

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF ELECTRICAL AND ELECTRONIC ENGINEERING



COURSE ASSIGNMENT

DATA STRUCTURES AND ALGORITHMS
PRIORITY QUEUE

Members	Vũ Đình Phúc – 202414778 Giang Văn Quang – 202414784 Phan Tiến Đạt – 202414691
Group	04
Class Code	161964
Name of instructor	Ph.D. Nguyễn Thanh Bình

Hanoi, 1 – 2026

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
LIST OF FIGURES.....	4
I. Introduction	5
1. Queue.....	5
1.1. Definitions.....	5
1.2. Basic Terminologies of Queue	5
1.3. Basic Operations	5
1.4. Real - world Applications.....	6
2. Priority Queue	6
2.1. Definitions.....	6
2.2. Key Characteristics	7
2.3. Real - world Applications.....	7
II. Operations	7
1. Basic operations	7
1.1. Enqueue	7
1.2. Dequeue.....	8
1.3. Peek	8
1.4. Empty	9
1.5. Size.....	9
2. Special Operations	9
2.1. Merge	9
2.2. Change Priority.....	10

2.3. Contains/find	11
2.4. Delete	11
III.Implementation	12
1. Implement Priority Queue using Unsorted Array (Sequential storage structure)	12
1.1. Overview	12
1.2. Operations.....	12
1.3. Time Complexity	14
2. Implement Priority Queue using Linked List (Linked storage structure).....	15
2.1. Overview	15
2.2. Operations.....	16
2.3. Time Complexity	17
3. Implement Priority Queue using Binary Heap (Linked storage structure).....	19
3.1. Binary Heap Overview	19
3.2. Operations.....	20
3.3. Time Complexity	26
IV.Applications.....	27
1. Overview	27
2. A Task Scheduler Implementation using Priority Queue	28
2.1. Problem.....	28
2.2. Solution	28
2.3. Specific operations.....	29

LIST OF FIGURES

Figure 1 First Come First Serve	5
Figure 2 Enqueue operation in Priority Queue	8
Figure 3 Dequeue operation in Priority Queue	8
Figure 4 Peek operation in Priority Queue	9
Figure 5 Empty operation in Priority Queue	9
Figure 6 Size operation in Priority Queue	9
Figure 7 Merge operation in Priority Queue	10
Figure 8 ChangePriority operation in Priority Queue	10
Figure 9 Contains operation in Priority Queue	11
Figure 10 Delete operation in Priority Queue	11
Figure 11 Inserted with a value and priority in a Array Priority Queue	13
Figure 12 Delete Element from Priority Queue in an Array Priority Queue	13
Figure 13 Peak operation in a Array Priority Queue	14
Figure 14 Linked Representation of Priority Queue	16
Figure 15 Enqueue operation in a Linked List Priority Queue	17
Figure 16 Dequeue operation in a Linked List Priority Queue	17
Figure 17 Representation of Binary Heap	19
Figure 18 Min Heap and Max Heap	20
Figure 19 Regular Queue vs. Priority Queue in Task Scheduling	29

I. Introduction

1. Queue

1.1. Definitions

A queue is a fundamental data structure that operates on the FIFO (First In First Out) principle.

FIFO Principle states that the first element added to the Queue will be the first one to be removed or processed. So, Queue is like a line of people waiting to purchase tickets, where the first person in line is the first person served.

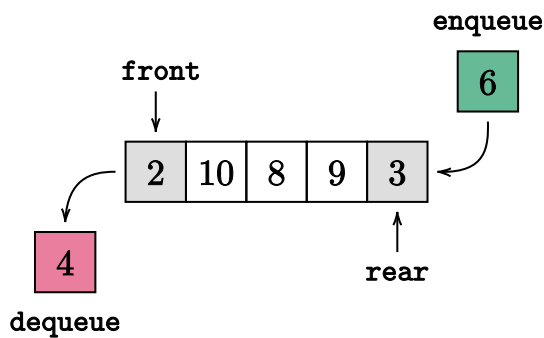


Figure 1 First Come First Serve

1.2. Basic Terminologies of Queue

front - The first element in the queue (next to be removed); also called head.

rear - The last element in the queue (most recently added); also called tail.

size - The current number of elements in the queue.

capacity - The maximum number of elements the queue can hold.

1.3. Basic Operations

enqueue - Inserts an element at the end of the queue.

dequeue - Removes and returns the element from the front of the queue.

getFront - Returns the front element without removing it.

getRear - Returns the rear element without removing it.

isEmpty - Checks if the queue contains any elements.

size - Returns the current number of elements in the queue.

1.4. Real - world Applications

Queues are essential for managing tasks sequentially on a “first-come, first-served” basis. For example, a print queue processes documents in the order they're submitted. Similarly, operating systems use queues to schedule processes waiting for the CPU, and in computer networks, they handle data packets as they arrive. This same principle applies to reservation systems, where customer requests are processed sequentially to ensure fairness.

2. *Priority Queue*

2.1. Definitions

A priority queue is a special type of queue where each element has an associated priority. Unlike a regular queue that operates on a FIFO (First In First Out) basis, a priority queue serves elements with the highest priority first.

If two elements have the same priority, their order is typically determined by when they were added to the queue (FIFO).

The best way to understand a priority queue is to think of an emergency room at a hospital.

- Regular Queue (FIFO): Like a line at a supermarket, the first person to arrive gets served first, regardless of their condition.
- Priority Queue: Patients are not treated based on arrival time. A patient with a severe injury (high priority) will be treated before someone with a minor scratch (low priority), even if the person with the scratch arrived earlier.

2.2. Key Characteristics

Priority-Based: The core principle is that elements with higher priority are dequeued before elements with lower priority.

2.3. Real - world Applications

Priority queues are essential for managing tasks based on importance rather than arrival time, allowing systems to handle the most critical operations first. For example, operating systems use them to let a CPU scheduler run vital system processes before low-priority tasks like a background download. Similarly, network routers prioritize critical data like a video call over less urgent traffic like a file download to ensure smooth performance. This principle is also key in AI algorithms like Dijkstra's, which uses a priority queue to efficiently explore the most promising options first when finding the shortest path, and in event simulations where tasks are processed based on their scheduled time.

II. Operations

1. *Basic operations*

1.1. Enqueue

Enqueue is an operation that add an element X to the rear of the queue based on its priority.

For example, given that the higher number has higher priority, when we perform the operation enqueue(1), the number 1 is inserted to the rear of the queue. After that, when operation enqueue(5) is performed, since $5 > 1$, the number “5” has higher priority and is inserted before number “1”. Since $1 < 3 < 5$, the number “3” is inserted after number 5 and before number 1 and so on.

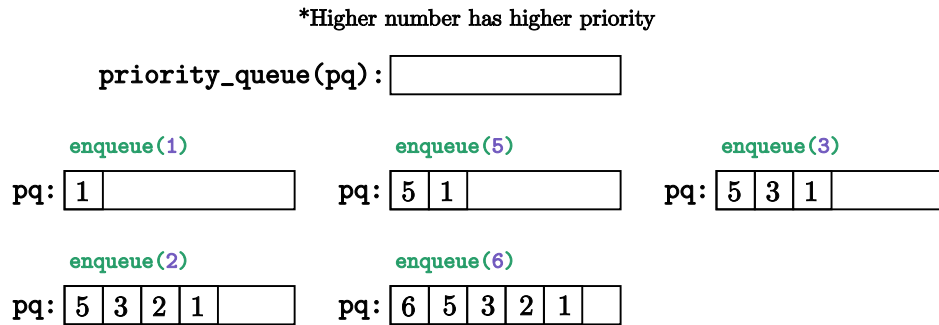


Figure 2 Enqueue operation in Priority Queue

1.2. Dequeue

Dequeue is an operation that removes the element from the front of the queue.

For example, given the condition that higher number has higher priority; firstly, the original queue is [6, 5, 3, 2, 1]. When we perform the dequeue() operation, the bigger number will be removed from the rear first. Therefore, the number “6” will be removed first and then number “5” and so on.

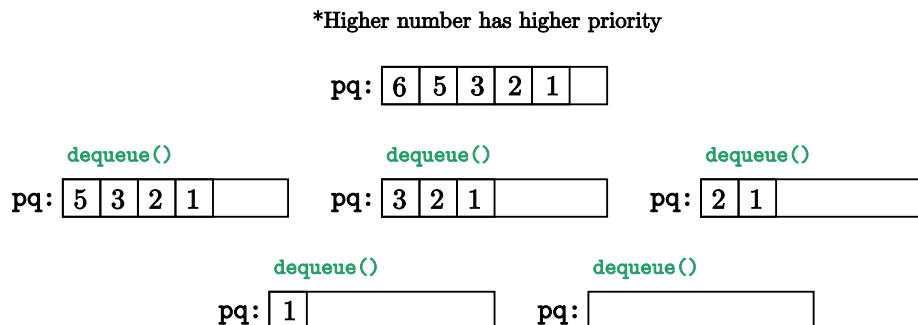


Figure 3 Dequeue operation in Priority Queue

1.3. Peek

Peek is an operation that returns the element with the highest priority in a priority queue.

For example, given the queue [6, 5, 3, 2, 1], the peek operation will retrieve the number “6” which has highest priority.

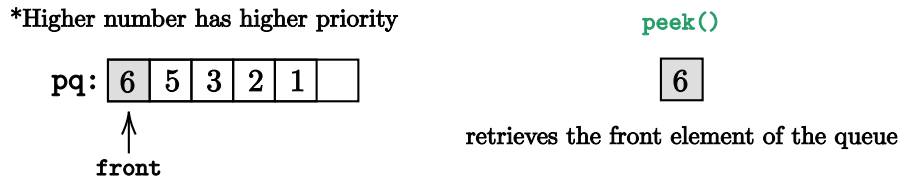


Figure 4 Peek operation in Priority Queue

1.4. Empty

The empty operation deletes all the elements in the queue and return an empty list.

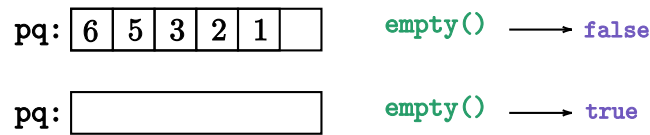


Figure 5 Empty operation in Priority Queue

1.5. Size

The size operation returns the size the queue.

For example, for the queue [6, 5, 3, 2, 1], the size operation will return the value 5 since it have 5 elements.

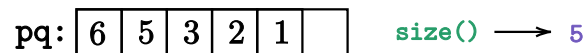


Figure 6 Size operation in Priority Queue

2. *Special Operations*

2.1. Merge

Merging in a priority queue is an advanced operation where you combine two separate priority queues into a single new priority queue while still maintaining the correct priority property (meaning the element with the highest priority is always at the front).

For example the merge operation of two priority queues. It takes all the elements from pq1 (containing [6, 5, 3, 2, 1]) and pq2 (containing [10, 7, 3, 1, 0]), and then combines them. Since the rule is “higher number has higher priority,” the resulting pq1 queue has been completely re-sorted into [10, 7, 6, 5, 3, 3, 2, 1, 1, 0], ensuring the largest element (number 10) is always at the front.

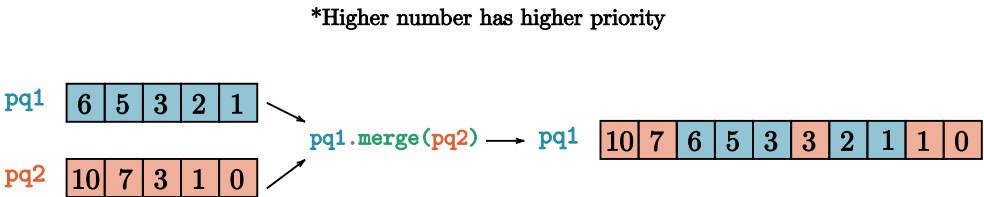


Figure 7 Merge operation in Priority Queue

2.2. Change Priority

This is an advanced operation that allows you to update the priority level of an element that is already inside the priority queue.

For example initially, 'S' has a priority value of 6. Following the rule “lower value has higher priority” (Min Priority Queue), the command `changePriority('S', 5)` updates and increases the priority of 'S'. Because the new value (5) has a higher priority than 'T's value (which is 6), the queue automatically reorganizes, moving 'S' to a position before 'T'.

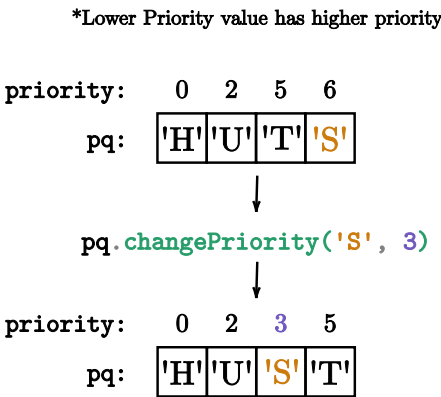


Figure 8 ChangePriority operation in Priority Queue

2.3. Contains/find

The contains or find operation in a priority queue is an action to check if a specific element currently exists within the queue.

The example shows a queue pq containing the elements [6, 5, 3, 2, 1]. When the contains(6) command is run, the system returns true because the number 6 is in the queue. Conversely, when contains(4) is run, the system returns false because the number 4 does *not* exist in the queue.

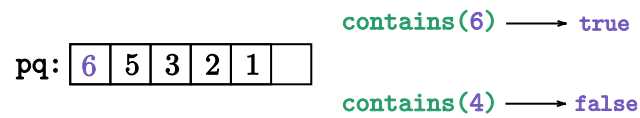


Figure 9 Contains operation in Priority Queue

2.4. Delete

The delete(element) operation in a Priority Queue is the action of removing a specific element from the queue, even if it is not the highest-priority element.

For example initially pq has 5 elements including [6, 5, 2, 1]. After the delete(3) command is implemented, the element 3 is removed, and the resulting pq queue is left with [6, 5, 2, 1].

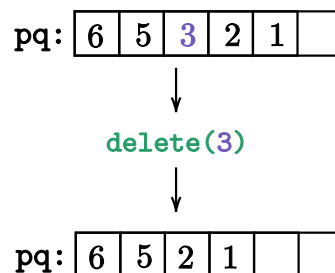


Figure 10 Delete operation in Priority Queue

III. Implementation

1. *Priority Queue using Unsorted Array (Sequential storage structure)*

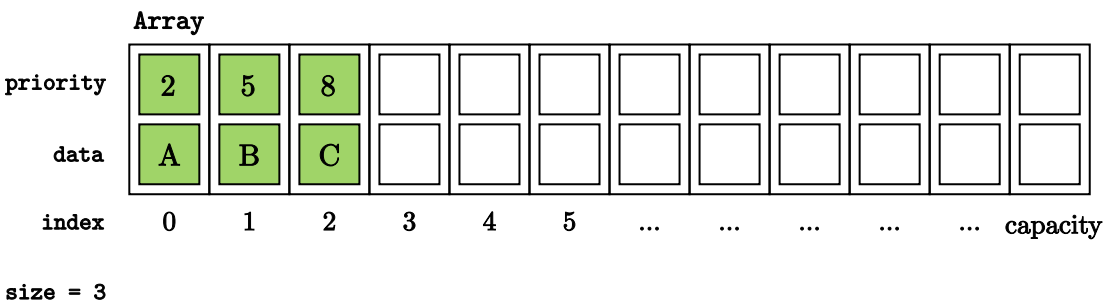
1.1. Overview

In an unsorted array-based priority queue, elements are not arranged by priority. To retrieve the element with the highest priority, the entire array must be searched, as the element with the lowest priority value (highest priority) can be located anywhere within the array.

1.2. Operations

1.2.1. Enqueue

This function inserts a new element into the queue by adding it to the end of the array. The element is appended without any sorting. The queue remains unsorted until the peek() or dequeue() function is called, which scans the entire array to find and handle the element with the highest priority based on the lowest priority value.



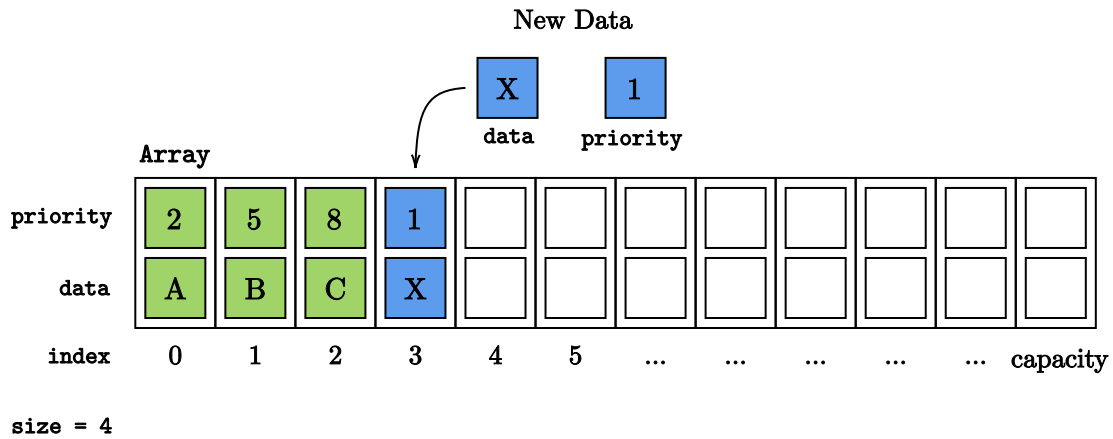


Figure 11 Inserted with a value and priority in a Array Priority Queue

1.2.2. Dequeue

This function removes and returns the element with the highest priority (i.e., the element with the lowest priority value). It locates the element with the highest priority using the `peek()` function, then removes it by shifting the remaining elements.

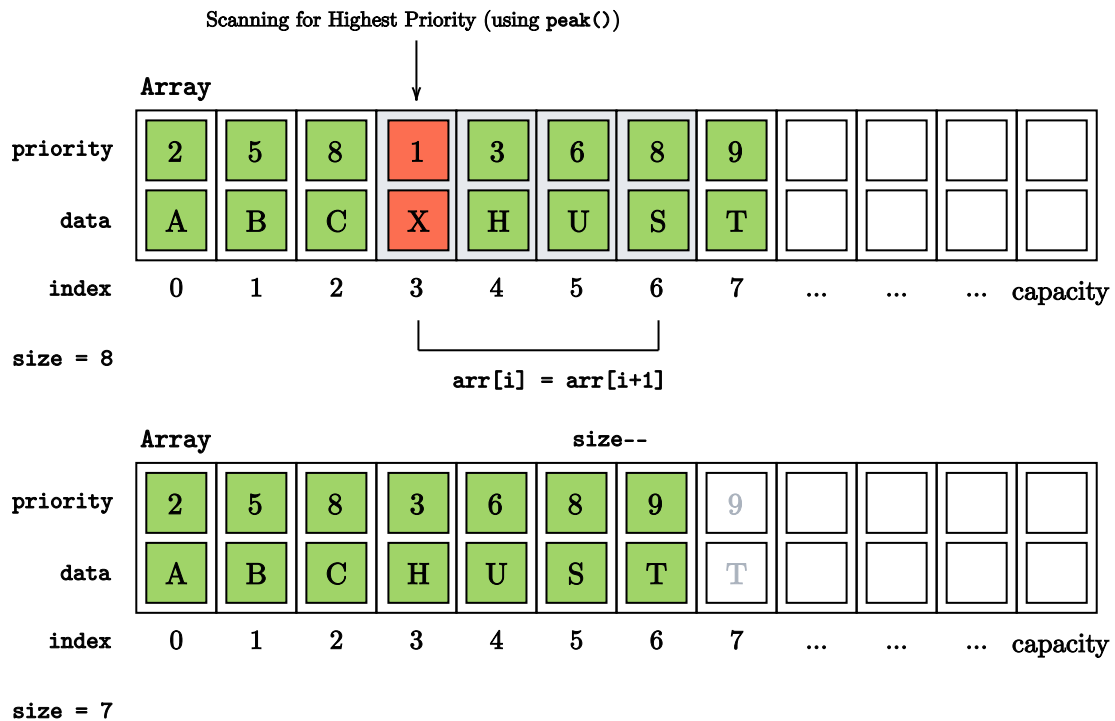


Figure 12 Delete Element from Priority Queue in an Array Priority Queue

1.2.3. Peek

This function returns the element with the highest priority (the element with the smallest priority value) without removing it from the queue. It scans the array to find the element with the lowest priority value and returns its index.

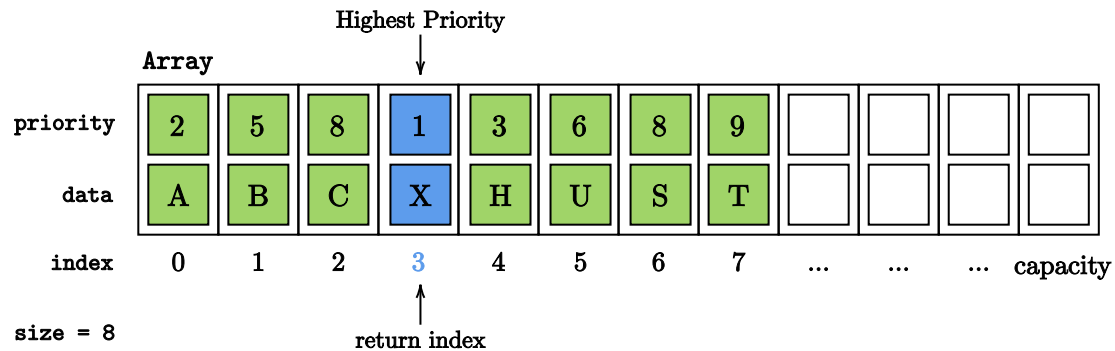


Figure 13 Peak operation in a Array Priority Queue

1.3. Time Complexity

1.3.1. Enqueue (Insertion)

Complexity: $O(1)$

Analysis: The enqueue operation simply appends the new element to the end of the array (at the current size index). Since the array does not need to be sorted or rearranged, this is a constant-time operation.

Note: If the array reaches its capacity and requires resizing (creating a larger array and copying elements), the amortized complexity remains $O(1)$, though the worst-case for that specific instance would be $O(N)$.

1.3.2. Peek (Find Highest Priority)

Complexity: $O(N)$

Analysis: Because the array is unsorted, the element with the highest priority could be located at any index. To guarantee the correct element is found, the algorithm must perform a linear search, visiting every element from index 0 to size-1.

1.3.3. Dequeue (Removal)

Complexity: $O(N)$

Analysis: This operation involves two expensive steps:

1. Search: It must first perform a linear scan to locate the highest-priority element, taking $O(N)$ time.
2. Shift: Once the element is removed, all subsequent elements must be shifted to the left to fill the gap and maintain contiguous storage. In the worst case (removing the first element), this also takes $O(N)$ time.

Consequently, the total time complexity is $O(N)$.

1.3.4. Auxiliary Space

Complexity: $O(1)$

Analysis: The algorithm operates in-place, requiring only a fixed amount of extra memory for temporary variables regardless of the input size N .

2. *Priority Queue using Linked List (Linked storage structure)*

2.1. Overview

A priority queue implemented using a linked list stores elements in a sequence of nodes. Each node contains the data, the priority value, and a pointer to the next node. Unlike the unordered array approach, in this linked list implementation, the elements are sorted according to priority at the time of insertion. The node with the highest priority (lowest priority value) is always maintained at the Head of the list.

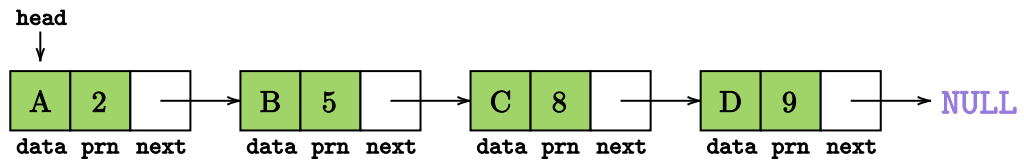
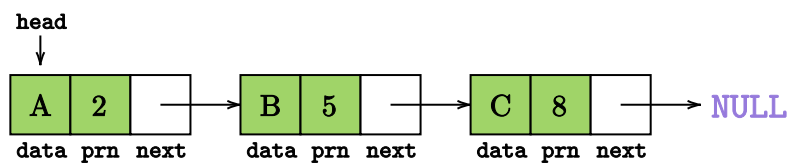


Figure 14 Linked Representation of Priority Queue

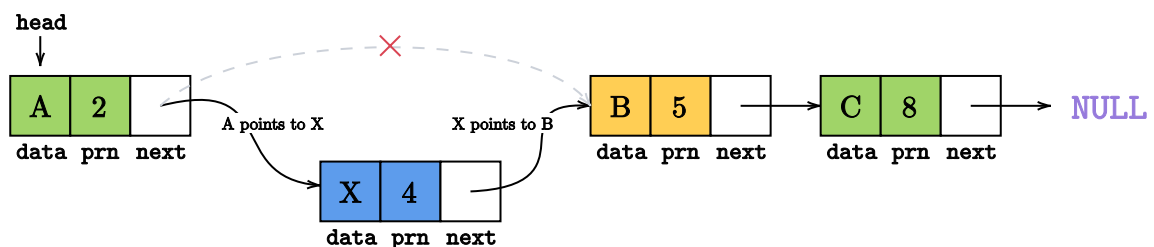
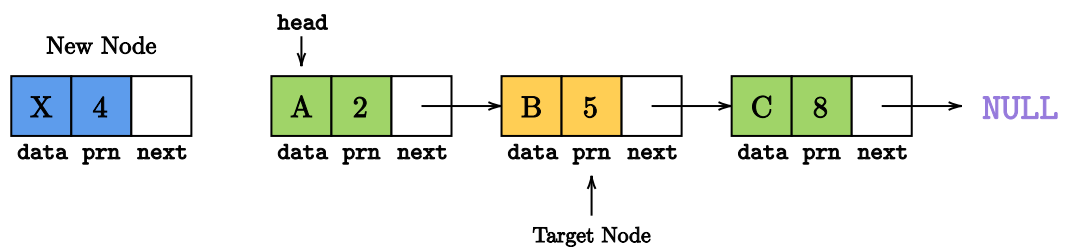
2.2. Operations

2.2.1. Enqueue

This function inserts a new element into the list while maintaining the sorted order. Instead of simply appending to the end, it traverses the list to find the correct position where the new element's priority is lower than (or equal to) the next node's priority. Pointers are then updated to insert the new node into this position.



Finding a node with a lower priority than the new element.



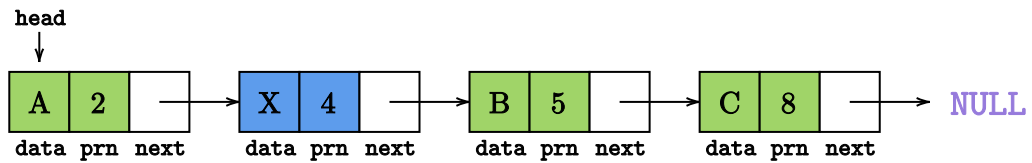


Figure 15 Enqueue operation in a Linked List Priority Queue

2.2.2. Dequeue

This function removes and returns the element with the highest priority. Since the list is strictly maintained in sorted order, the highest priority element is always located at the Head of the list. Therefore, this operation simply removes the first node and updates the Head pointer to the next node, which is significantly faster than searching an array.

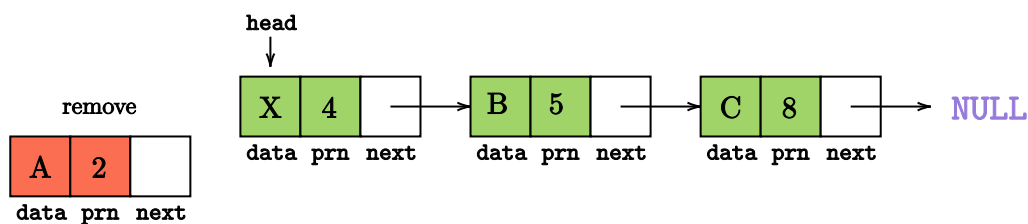


Figure 16 Dequeue operation in a Linked List Priority Queue

2.3. Time Complexity

2.3.1. Enqueue (Insertion)

Complexity: $O(N)$ (Linear Time)

Analysis: Since the list must remain sorted by priority, inserting a new element requires traversing the list to find the correct insertion position.

- **Best Case:** The new element has the highest priority and is inserted at the Head ($O(1)$).
- **Worst Case:** The new element has the lowest priority, requiring a traversal to the very end of the list ($O(N)$).

- **Average Case:** The element is inserted somewhere in the middle, which is still proportional to N .

2.3.2. Dequeue (Removal)

Complexity: $O(1)$ (Constant Time)

Analysis: Because the list is maintained in sorted order, the element with the highest priority is always located at the Head of the list. Removing this element simply involves updating the Head pointer to the next node and freeing the memory. No traversal or shifting of elements is required.

2.3.3. Peek (Find Highest Priority)

Complexity: $O(1)$ (Constant Time)

Analysis: Similar to Dequeue, accessing the highest-priority element is instantaneous because it is always residing at the Head node. We simply return the data stored in the first node.

2.3.4. Auxiliary Space

Complexity: $O(1)$

Analysis: The algorithm uses a constant amount of extra memory for temporary pointers (e.g., current, previous, newNode) needed for traversal and linking. While the storage space grows linearly with inputs ($O(N)$), the *auxiliary* space for operations remains constant.

3. *Priority Queue using Binary Heap (Linked storage structure)*

3.1. Binary Heap Overview

3.1.1. Fundamentals

A Binary Heap is a special type of complete binary tree, meaning all levels are filled except possibly the last, which is filled from left to right.

It follows the Heap Property: The ordering relationship between parent and child is strict.

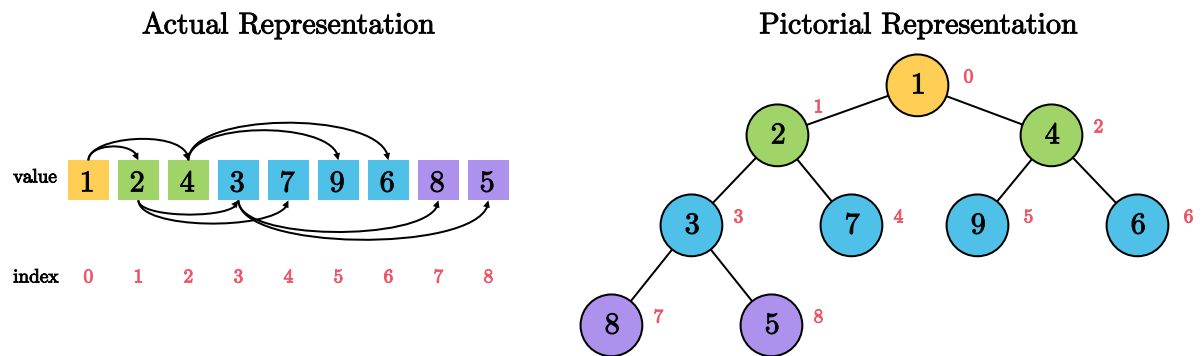


Figure 17 Representation of Binary Heap

3.1.2. Classification

Min Heap: The value of the root node is the smallest, and this property is true for all subtrees.

Max Heap: The value of the root node is the largest, and this rule also applies to all subtrees.

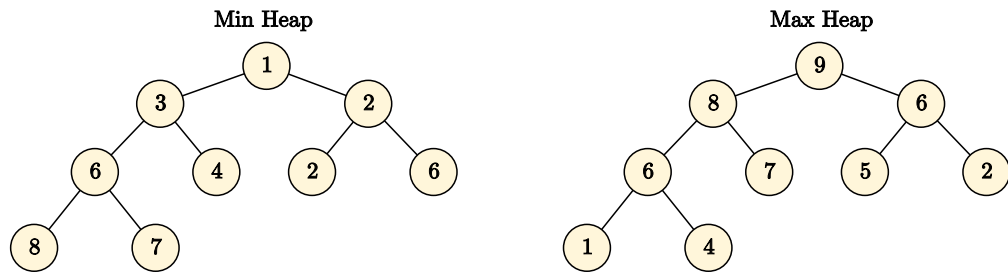


Figure 18 Min Heap and Max Heap

3.1.3. Storage

A Binary Heap is a Complete Binary Tree. A binary heap is typically represented as an array.

- The root element will be at `arr[0]`.
- Formula to access the node at index i :
 - Left child: $2i + 1$
 - Right child: $2i + 2$
 - Parent: $(i-1)/2$

3.2. Operations

3.2.1. Insert

Inserting an element into a heap is a systematic process that ensures the heap property is maintained. This process involves a series of steps that guarantee the min-heap rule is upheld when a new element is added.

Procedure for Insertion:

1. Placement of the New Element:

- The new element is initially inserted as the last element in the tree structure.
- If the tree is empty, the new element becomes the root node.
- If the lowest level of the tree is not fully populated, the new element is placed next to the last node at that level.

- If the lowest level is complete, the new element is appended as a child of the first node at that level.

2. Heap Property Maintenance:

- After insertion, the parent node of the new element is compared with the new element.
- If the parent node is greater than the new element, a swap is performed between the two nodes.
- This swapping continues until the parent node is less than or equal to the new element, thereby adhering to the min-heap rule.

Array Representation:

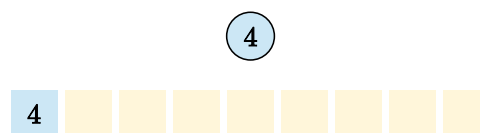
In the context of an array representation of a heap, the insertion process simplifies to placing the new element in the first available position in the array. The subsequent steps ensure that the heap property is restored by comparing and swapping the new element with its parent as necessary.

Inserting into the Min-Heap: Example

The process of populating a min-heap-based priority queue using the sample values: 4, 7, 3, 8, 2, 9, 6, 5, and 1. Each step of the process is illustrated through both tree and array representations of the min-heap.

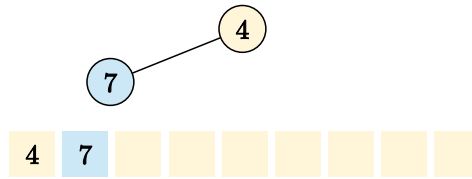
1st Element – Inserting the 4 into an Empty Priority Queue

The first element to be inserted becomes the root node of the tree; in the array, we place it at the first position:



2nd Element – Inserting the 7

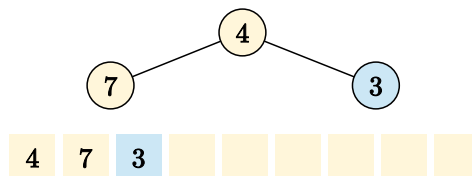
We append the 7 below the first node of the lowest level – that is, below the root on the left. In the array, we simply append it:



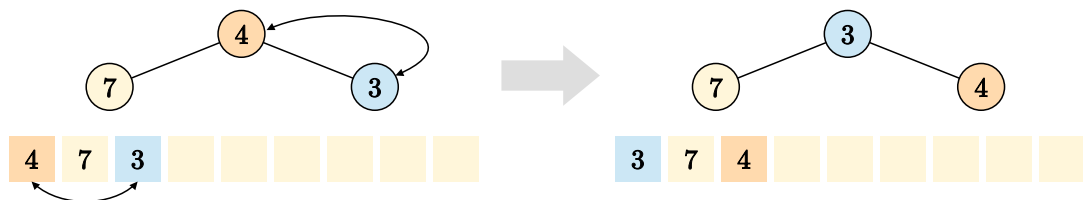
The 7 is greater than its parent node 4 – thus, the insertion operation is complete. The smallest element is still at the beginning of the priority queue.

3rd Element - Inserting the 3

We append the 3 next to the last node of the lowest level, that is, as right child of the 4. In the array, it comes at the end:



The 3 is less than its parent node. The min-heap rules are, therefore, violated. We restore the min-heap by swapping the 3 with the 4:

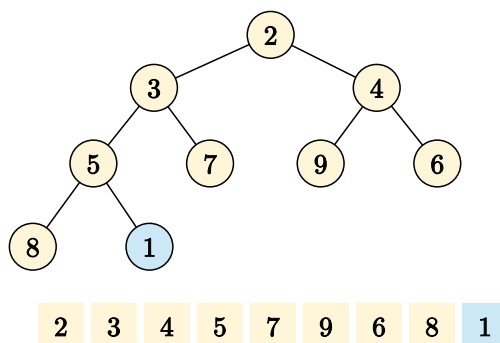


We now have a valid min-heap again.

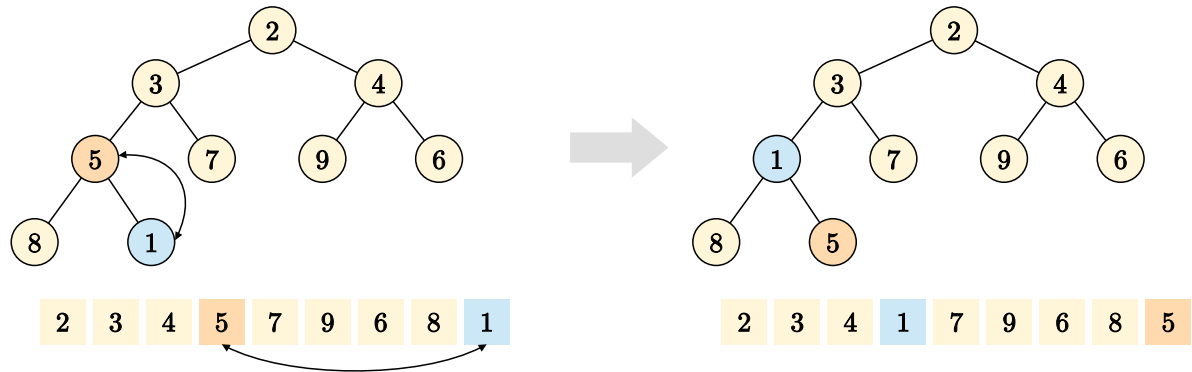
We skip 8, 2, 9, 6, and 5 (these are inserted analogously) and come to the...

9th Element – Inserting the 1

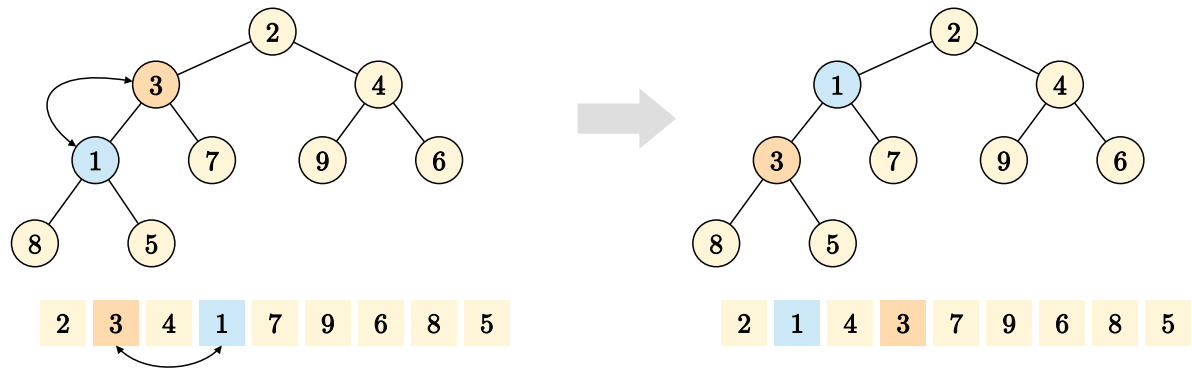
Finally, we add the 1 to the end of the queue (and the array):



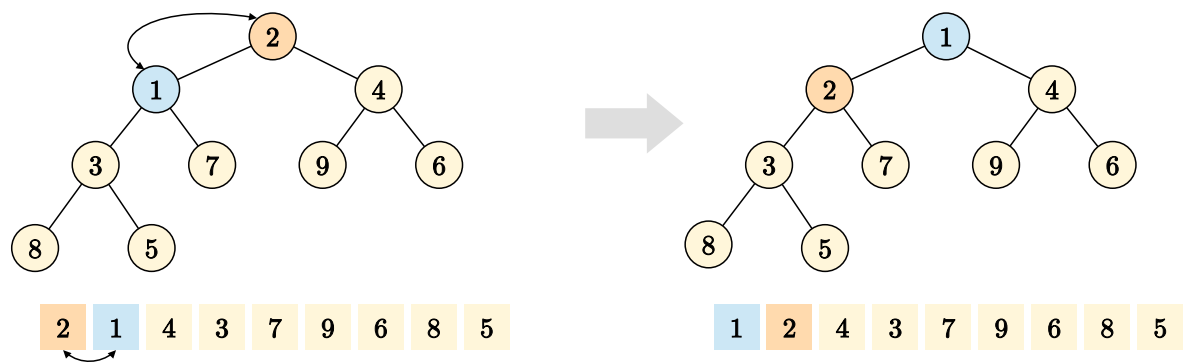
The 1 is greater than its parent node 5; thus, our tree is no longer a valid min-heap. To fix it, we first swap the 1 with the 5:



The 1 is also greater than its new parent node 3; thus, we swap again:



The 1 is also greater than the root 2, so we swap a third time:



Since the 1 has now reached the root, the operation is finished. The tree is again a min-heap. The smallest element is at the tree's root (and at the beginning of the array).

This reaching up of the inserted element in the way just shown is called “sift up”.

3.2.2. Remove

In a min-heap data structure, the smallest element is consistently located at the root of the tree or at the beginning of the array. The process for removing this smallest element involves the following steps:

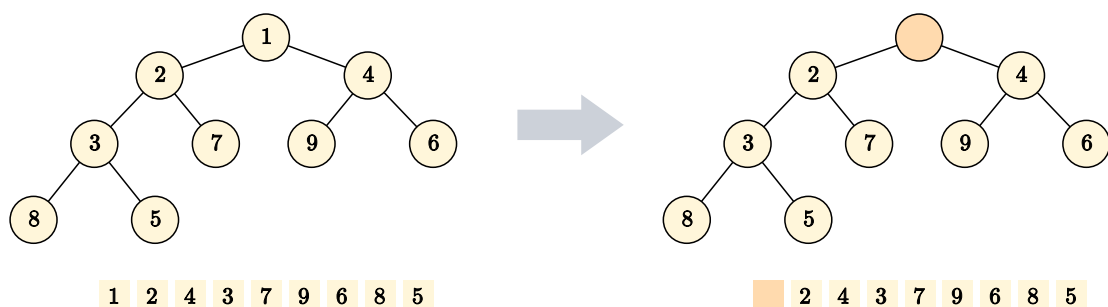
- 1. Root Removal:** The root element of the tree is removed.
- 2. Node Replacement:** The last node from the lowest level of the tree is moved to occupy the position vacated by the root. In an array representation, this corresponds to the last element.
- 3. Heap Property Restoration:** The node that has been moved is compared with its children. If it is greater than any of its children, it violates the min-heap property. In such cases, the node is swapped with its smallest child to restore the heap property.

This procedure ensures that the min-heap structure is maintained after the removal of the smallest element.

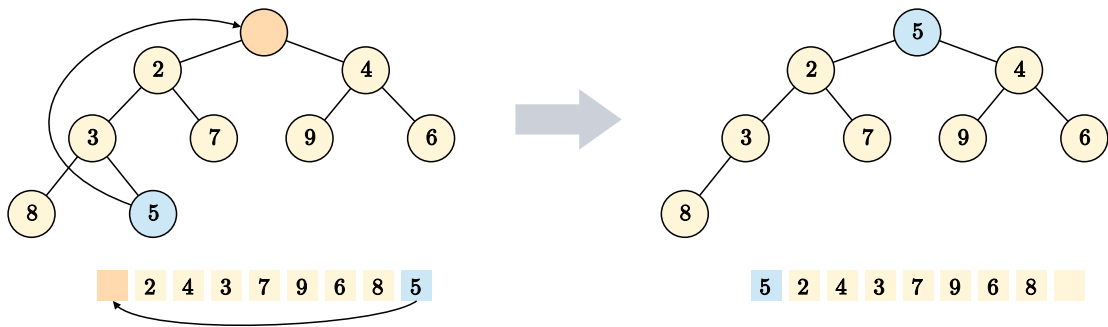
Removing from the Min-Heap: Example

The following example shows how we remove the root element of the min-heap filled in the last chapter – and then restore the min-heap condition.

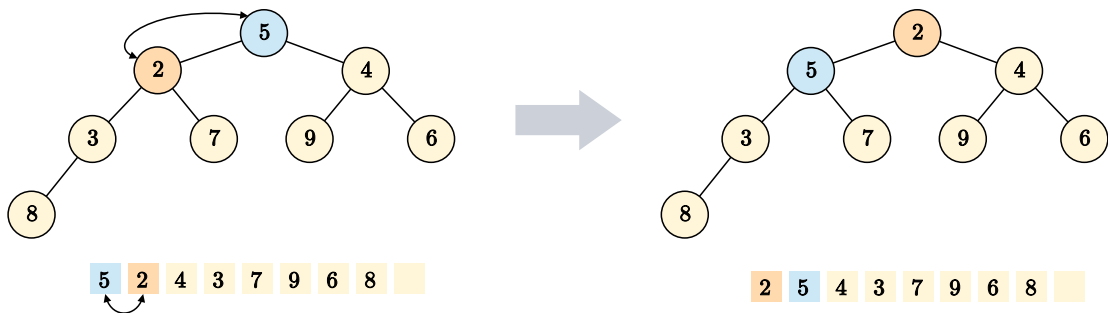
The first thing we do is take out the root element:



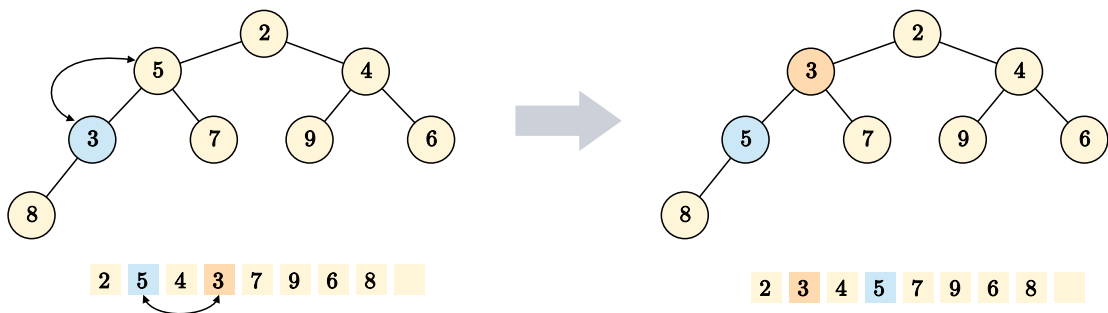
Second, we move the tree's last element, the 5, to the now-vacated root node:



Since the new root element, 5, is greater than the smallest of its children, 2, we swap those two elements:



The 5 is still greater than the smallest of its children, the 3. We swap a second time:



The 5 is now greater than its only child; we have thus restored the min-heap condition.

The root of the min-heap (the first field of the array) now contains the 2, the new smallest element after removing the 1.

The reaching down of the element moved to the root is called "sift down".

3.3. Time Complexity

The efficiency of Binary Heap operations is strictly determined by its structural property as a Complete Binary Tree. Unlike a standard linked list where operations might require traversing N elements, a Binary Heap leverages its tree height to reduce the number of comparisons significantly.

3.3.1. Relationship between Tree Height and Size

Since a Binary Heap is a complete binary tree, its size N grows exponentially with its height h ($N \approx 2^h$). Conversely, the height is logarithmic relative to the number of elements:

$$h \approx \log_2 N$$

3.3.2. Enqueue (Insertion) Complexity

The enqueue operation consists of two main steps:

1. **Insertion:** Adding the new element to the end of the array (leaf node). This takes $O(1)$.
2. **Restoration (Sift Up):** The new element is compared with its parent and swapped if the heap property is violated.
 - **Best Case:** The new element is correctly placed immediately (no swap needed). Complexity: $O(1)$.

Worst Case: The new element has the highest priority (e.g., smallest value in a Min-Heap) and must be swapped repeatedly from the leaf node up to the root.

- The maximum number of swaps is equal to the height of the tree (h).
- Therefore, the worst-case complexity is $O(h) = O(\log N)$.

3.3.3. Dequeue (Removal) Complexity

The dequeue operation removes the root element (highest priority) and involves:

1. **Replacement:** Moving the last element of the heap to the root position. This takes $O(1)$.
2. **Restoration (Sift Down):** The new root is compared with its children and swapped with the higher-priority child until the heap property is restored.

Worst Case: The element moved to the root violates the heap property at every level and must be swapped down to the deepest leaf level.

- Similar to insertion, the path of the element traverses the height of the tree.
- The maximum number of comparisons/swaps is proportional to h .
- Therefore, the worst-case complexity is $O(h) = O(\log N)$.

4. *Comparsion*

Here is the comparison table between the different implementations described above of priority queue. It is easy to see that the binary heap is the most efficient implementation for priority queue.

	Unsorted Array	Linked List	Binary Heap
Enqueue	$O(1)$	$O(N)$	$O(\log N)$
Dequeue	$O(N)$	$O(1)$	$O(\log N)$
Peek	$O(N)$	$O(1)$	$O(1)$

IV. Applications

1. *Overview*

Priority queues have many practical applications across computing: scheduling processes for operating systems, Dijkstra algorithm to find the shortest path (select optimal node, Huffman algorithm for data compression,...

Priority queues are especially helpful for scheduling task by processing them in order. Here, we will demonstrate how to implement a task scheduler using priority queue to complete tasks with different levels of importance.

2. A Task Scheduler Implementation using Priority Queue

2.1. Problem

Suppose that you have a long list of tasks to finish. For example, a list of assignments with different deadlines or simply a list of groceries item with different urgency,... Therefore, you need a to-do list task scheduler to organize a list of tasks, each with a different priority, deadline, or importance level, and determine which task should be completed next. The problem is to always retrieve the most urgent or highest-priority task efficiently while allowing users to add, update, or remove tasks at any time.

2.2. Solution

We will implement the mentioned behaviour using priority queue

- The priority queue will be implemented using binary heap. The behaviour of our priority queue is the same as the implementation above. Each node will contain:
- A string type value to represent the task.
- A int type priority to represent the the associated priority.

As mentioned above, the binary heap implementation will be the same.

- Input: the input will be a list of different tasks from the user with their associated priority. (deadlines, level of urgency...)
- Output: A task scheduler that maintain the order automatically, ensuring that the most important tasks are handled first. This perfect for priority queue.

A priority queue is like a special to-do list where each task has its own level of importance. Unlike a regular to-do list, where tasks are done in the order they are added, a priority queue makes sure the most important task gets done first.

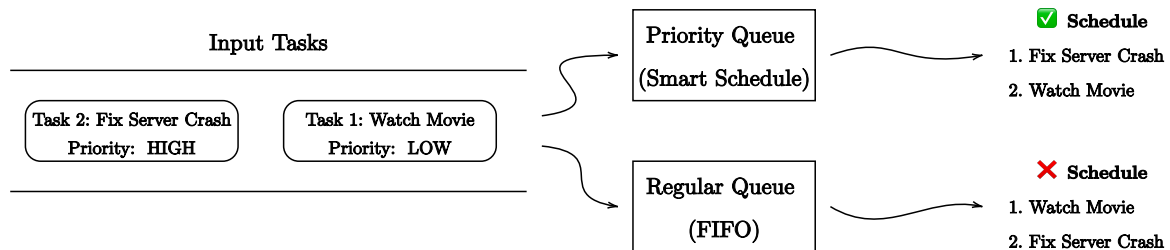


Figure 19 Regular Queue vs. Priority Queue in Task Scheduling

2.3. Specific operations

From the description above, we will associate the real world action with the following operations of priority queue for our task scheduler.

- **Adding Tasks (Enqueue):** When you add a task to a priority queue, you also say how important it is. The more important, the sooner it gets done.
- **Doing Tasks (Dequeue):** The priority queue takes care of doing the most important task first. It's like automatically picking the task with the highest priority.
- **Checking the top task (Peek):** You can peek at the top task without actually doing it. This is useful when you want to see what's the most important thing right now.
- **Changing Importance (Change Priority):** If something becomes more urgent or less critical, you can change its priority in the queue.

In summary, a priority queue is like having a smart to-do list that always helps you focus on the most important things first – a helpful concept in the busy world of computing and problem-solving.

3. *Result*

The users will type each task that they want to do in the “input_task.txt” file with their associated priority. Then, the program will write all the tasks in the “output_task.txt” file. When “DO_TASK” is typed in the input file, the program will automatically call the task with the highest priority and output as “Doing Task: (name of task)” in the output file. That task will also be removed from the list and the program will proceed with the next highest priority task.

input_task.txt	output_task.txt
ADD_TASK FixServer 1	Added Task: FixServer (Priority: 1)
ADD_TASK WatchMovie 10	Added Task: WatchMovie (Priority: 10)
ADD_TASK WriteCode 5	Added Task: WriteCode (Priority: 5)
DO_TASK	Doing Task: FixServer
DO_TASK	Doing Task: WriteCode